

IMAGE PROCESSING & COMPUTER VISION
GITHUB, GOOGLE COLLABORATORY, READING, AND VIEWING IMAGE



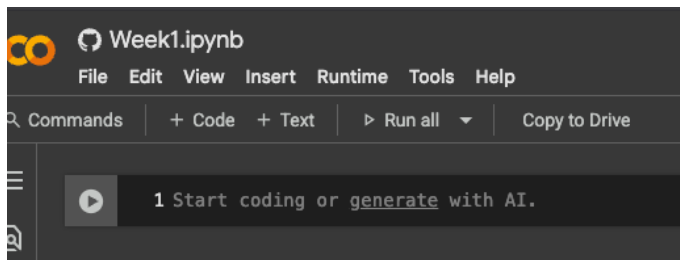
KHEN MUHAMMAD CAHYO

244107023002

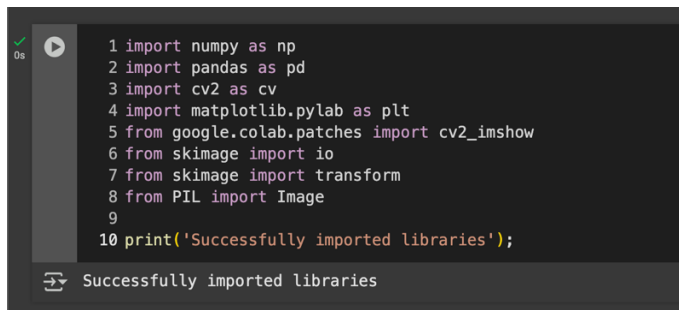
TI-3I

STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT

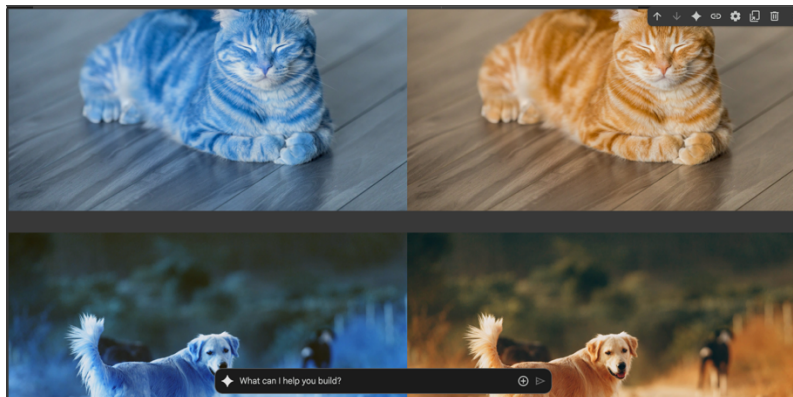
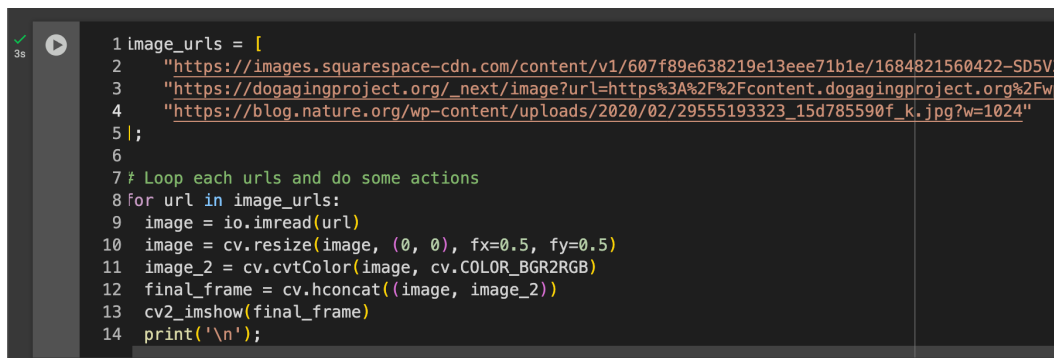
PRACTICUM – Connecting GitHub to Google Collab



PRACTICUM – Importing Library in Python



PRACTICUM – Reading & Showing Image in Python



PRACTICUM – Showing Image Resolution

```
16
17 # Print Image Size
18 height = image_2.shape[0]
19 width = image_2.shape[1]
20 print('Image resolution: height * width = ', height, '*', width);
21 print('\n')
```

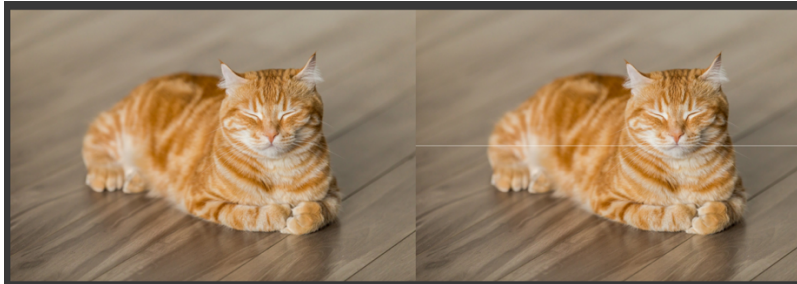


Image resolution: height * width = 834 * 1250

PRACTICUM – Diagonal Line on Image

```
# Drawing White Diagonal Line
image_3 = cv.cvtColor(image, cv.COLOR_BGR2RGB)
for y in range (width):
    image_3[int((height) / 2), y] = [255, 255, 255]

final_frame2 = cv.hconcat((image_2, image_3))
cv2_imshow(final_frame2)
print('\n');
```



QUESTION

Answer Number 1

Because Google Collab provide us Python environment there especially on Machine Learning, and Data Analytic fields. Most of popular Python modules are already installed too, if any modules are not installed by Google Collab we can just install it by ourself using **!pip install module-name**.

Answer Number 2

In this lab, each library has its own role. NumPy and Pandas are not actually used directly in the image manipulation process, but are typically used for processing numerical or tabular data related to images. OpenCV (**cv2**) plays an important role in reading, resizing, converting colors, and combining images. Matplotlib is used for visualization, although here image display is mostly handled by Google Colab's **cv2_imshow**, which allows images from OpenCV to be displayed directly in the notebook. Meanwhile, **skimage** (scikit-image) is used to read images from URLs with `io.imread` and provides various transformation functions, while **PIL** (Python Imaging Library) is typically used for simple image manipulation.

Not all libraries are required to be used in this practicum, as some of them are only complementary or alternative. For example, **PIL** and **Pandas** are not essential if all manipulations can be handled with OpenCV and skimage. However, the presence of many libraries provides flexibility so that practitioners can compare how each one works and choose the tools that best suit their needs.

Answer Number 3

The code is used to reduce the image size to half of its original dimensions in both width and height with a scale factor of 0.5. The purpose is to make the image lighter to display, process, or compare, especially if the original image has a high resolution that can cause slow display or memory consumption. If this command is not executed, the image will remain at its original resolution, requiring more memory and processing time, and potentially making the visualization too large for the screen or output in Google Colab.

Answer Number 4

The code `[255, 255, 255]` represents the color white in RGB format, where each red, green, and blue channel value is at maximum intensity (255). In that line of code, this value is assigned to each pixel along the horizontal line in the center of the image, creating a clearly visible white line. If a different color value is used, for example `[255, 0, 0]`, then the line formed will be red, so the use of `[255, 255, 255]` specifically aims to mark or draw a white line on the image.

Answer Number 5

Image resolution is determined by the number of pixels that make up the image, so the more pixels there are, the higher the resolution. A pixel is the smallest unit of a digital image that stores color information, so in high-resolution images, every detail of an object is represented by more pixels, resulting in a sharper and clearer display. Conversely, low-resolution images have fewer pixels, so the details of objects appear blurry, broken, or unclear because one pixel must represent a larger area of the image.

TASK – Drawing Vertical Line

```
12 # Print Image Size
13 kangoro_line_height = kangoro_line_img.shape[0]
14 kangoro_line_width = kangoro_line_img.shape[1]
15 print('\n')
16 print('Image resolution: height * width = ', height, '*', width);
17
18 # Drawing Vertical Line
19 for x in range(kangoro_line_height):
20     kangoro_line_img[x, int(kangoro_line_width / 2)] = [255, 255, 255]
21
22 kangoro_frame = cv.hconcat((kangoro_img, kangoro_line_img))
23 cv2_imshow(kangoro_frame);
```



TASK – Drawing Combination Line

```
# Drawing Combination Line
for i in range(kangoro_line_height):
    x = int(i * kangoro_line_width / kangoro_line_height)
    kangoro_line_img[i, x] = [255, 255, 255]

for i in range(kangoro_line_height):
    x = int(kangoro_line_width - (i * kangoro_line_width / kangoro_line_height) - 1)
    kangoro_line_img[i, x] = [255, 255, 255]
```



TASK – Drawing Middle Line

```
31 # Drawing Middle Line
32 for y in range(int(kangoro_line_width / 4), int(3 * kangoro_line_width / 4)):
33     kangoro_line_img[int(kangoro_line_height / 2), y] = [255, 255, 255]
```



TASK – Drawing Box

```
# Drawing Box
for x in range(int(kangoro_line_height / 4), int(3 * kangoro_line_height / 4)):
    for y in range(int(kangoro_line_width / 4), int(3 * kangoro_line_width / 4)):
        kangoro_line_img[x, y] = [255, 255, 255]
```

